

# EU Product Liability Directive Kit — Directive (EU) 2024/2853 for software teams

---

- [00 — Start here](#)
  - [What this kit is](#)
  - [Why that particular piece of work](#)
  - [Who this is for](#)
  - [Who this is NOT for](#)
  - [What is NOT in this kit, deliberately](#)
  - [How to use this in an afternoon](#)
  - [What the free pld-watch tool does — stated honestly](#)
  - [Primary sources used](#)
  - [A note on the date, because it moved](#)
  - [Files in this kit](#)
  - [How to keep this current](#)
- [01 — Am I in scope?](#)
  - [Before you start: the directive does not work like a compliance regime](#)
  - [Question 0 — The date. What does the new regime attach to?](#)
  - [Question 1 — Is what you ship a “product”?](#)
  - [Question 2 — Which role do you play?](#)
  - [Question 3 — Do you place it on the EU market?](#)
  - [Question 4 — The free and open-source software carve-out](#)
  - [Question 5 — Could anyone actually claim against you?](#)
  - [Your scope card](#)
  - [When to stop and get a lawyer](#)
- [02 — What changes from 1985](#)
  - [The one-paragraph version](#)
  - [Side by side](#)
  - [The five changes that matter to an engineering team](#)
  - [What did NOT change](#)
  - [What this means for your roadmap](#)
- [03 — Update supportability evidence](#)
  - [What this is for, in one paragraph](#)
  - [Why the record, specifically](#)
  - [The five artefacts](#)
  - [Artefact 1 — Component inventory](#)
  - [Artefact 2 — EOL register, and where pld-watch fits](#)

- [Artefact 3 — Update policy with stated support windows](#)
- [Artefact 4 — End-of-support communication](#)
- [Artefact 5 — Recording a decision NOT to update](#)
- [Retention: how long to keep all of this](#)
- [The afternoon: a concrete sequence](#)
- [What “good” looks like after six months](#)
- [04 — Support policy templates](#)
  - [How to use these](#)
  - [Template A — Public security update support policy](#)
  - [Template B — End-of-support notice](#)
  - [Template C — Component end-of-life register](#)
  - [Template D — Internal record: decision not to update](#)
  - [Bonus — Supplier identification record \(distributors and resellers\)](#)
  - [Before any of this goes out](#)
- [05 — Board brief](#)
  - [1. The change, in five lines](#)
  - [2. What it is not — read this before reacting](#)
  - [3. Our exposure, honestly assessed](#)
  - [4. What it implies for engineering](#)
  - [5. Why this is worth doing now rather than later](#)
  - [6. Decisions we are asking the board for](#)
  - [7. Insurance](#)
  - [8. Open questions we cannot answer internally](#)
  - [9. Bottom line](#)

# 00 — Start here

---

## EU Product Liability Directive (EU) 2024/2853 — Update Supportability Kit

Companion to the free `pld-watch` tool — <https://github.com/ntoledo319/pld-watch>

Kit version 1.0. Written against the directive text as published in the Official Journal on 18 November 2024 and as corrected by Corrigendum 2026/90364 of 7 May 2026. Last checked against EUR-Lex: 14 September 2026.

---

**Not legal advice.** This kit is engineering and governance material written from the primary text of Directive (EU) 2024/2853. It is not legal advice, it is not a certification, and it is not a compliance determination. Nothing here tells you whether you are in scope, whether your product is defective, or whether you are liable. Those are decided by a court on the facts, under the national law that transposes the directive. Take advice from a qualified lawyer in the relevant Member State before relying on anything in this kit.

---

## What this kit is

Six documents that help a software team do one specific, bounded piece of work:

**Build and keep a defensible record that you can, and do, supply security updates for the software you ship — and that when you decide not to, you decide it deliberately and write it down.**

That is not the whole of the directive. It is the part that changed most for software teams, the part that is squarely inside engineering's control, and the part that is cheap to do now and expensive to reconstruct later.

## Why that particular piece of work

Under the new directive, a manufacturer can normally escape liability by proving the defect did not exist when the product was placed on the market, or came into being afterwards (Article 11(1)(c)). That defence is switched off in four situations where the thing is within the manufacturer's control — and one of them is:

“a lack of software updates or upgrades necessary to maintain safety” —  
Article 11(2)(c)

And a product is treated as remaining within your control simply because you **have the ability** to supply software updates, yourself or via a third party (Article 4(5)(b)).

So the old instinct — *we shipped it clean, whatever happened later is not on us* — no longer works the same way for software you can still update.

**Read this next, because it is the honest other half:** the directive says in terms that it “does not impose any obligation to provide updates or upgrades for a product” (Recital 51). There is no patching mandate here. What there is, is a liability consequence attached to a *defect that consists in* the absence of a safety-necessary update, where you had control. Those are different things, and a kit that blurs them would be selling you a fear that the text does not support.

## Who this is for

- Engineering leaders and CTOs at companies shipping software, SaaS, firmware or AI systems that reach users in the EU.
- Technical founders who need to answer an investor, insurer or enterprise customer question about PLD readiness.
- Product and platform teams who own release, support and deprecation policy.

It assumes you can read a `package.json`, run a CLI, and write a policy page. It does not assume any legal background.

## Who this is NOT for

- Anyone looking for a compliance certificate, a badge, or a statement that they are “PLD compliant”. No such thing exists. The directive creates no certification scheme, no notified bodies, no registration, and no regulator that approves you. There is nothing to be certified against.
- Anyone who needs a scope determination. Whether *your* product is in scope is a legal question about your facts. File 01 helps you frame it. It cannot answer it.
- Medical device, automotive, aviation or other regulated-sector teams looking for sector guidance. Those sectors carry their own safety law, and the interaction is beyond this kit.

## What is NOT in this kit, deliberately

- **Member State transposition detail.** The directive had to be transposed into national law by 9 December 2026 (Article 22(1)). Transposition varies, and some things — standard of proof, disclosure procedure, non-material damages, whether a state dropped the development-risk defence under Article 18 — are set nationally. We do not guess at any of it. Where it matters, you will see a [VERIFY: ...] marker.
- **Any prediction of outcomes.** No claim here that doing this work makes you safe, wins a case, or reduces a premium.
- **Contract drafting.** Supplier terms, indemnities and the Article 12(2) recourse waiver are lawyer work.

## How to use this in an afternoon

Roughly three to four hours for one product, with one engineer and one person who can make policy decisions.

| Step                                                                  | Time   | File                                 | Output                                                                           |
|-----------------------------------------------------------------------|--------|--------------------------------------|----------------------------------------------------------------------------------|
| 1. Work out whether the directive plausibly reaches you               | 30 min | 01-AM-I-IN-SCOPE.md                  | A filled-in scope card and a written list of the questions you need a lawyer for |
| 2. Brief yourself on what actually changed                            | 30 min | 02-WHAT-CHANGES-FROM-1985.md         | Enough context to explain it to your team without overstating it                 |
| 3. Run <code>pld-watch scan --json</code> and start the evidence file | 60 min | 03-UPDATE-SUPPORTABILITY-EVIDENCE.md | A component inventory with EOL status, dated and stored                          |
| 4. Draft the support policy and the EOL register                      | 60 min | 04-SUPPORT-POLICY-TEMPLATES.md       | Draft policy, draft register, ready to go to legal review                        |
| 5. Write the board note                                               | 20 min | 05-BOARD-BRIEF.md                    | A two-page brief for a board, investor or insurer                                |

If you only have one hour: do step 3. A dated, honest component inventory is the single

artefact that is hardest to recreate after the fact and easiest to build today.

## What the free `pld-watch` tool does — stated honestly

`pld-watch` answers one narrow question:

Of the runtimes and frameworks this project *declares*, which ones are past their upstream end-of-life date?

It reads `.nvmrc`, `.python-version`, `go.mod`, `package.json` (the engines block plus a built-in map of known frameworks), `requirements.txt` / `requirements-dev.txt`, and `FROM` lines in a `Dockerfile`. It checks each version it finds against the public **endoflife.date** catalogue and prints the ones that are past EOL, near EOL, supported, or unknown. `--json` gives you the same result in a machine-readable form for CI. Exit code 1 means at least one component is past EOL, 0 means none are, 2 means it found nothing to check.

That matters because an end-of-life component cannot receive an upstream security update at all. If a safety-relevant vulnerability lands in it, there is no patch to ship. That is a fact worth knowing deliberately rather than discovering during an incident.

## What `pld-watch` does not do, and cannot do

- It does **not** tell you whether you are in scope of the directive.
- It does **not** tell you whether your product is defective. Defectiveness is decided by a court on all the circumstances (Article 7). No scanner decides it.
- It does **not** tell you whether you are liable.
- It does **not** check for known vulnerabilities. It is not an SCA or CVE scanner. A supported version can still be vulnerable; an EOL version may have no known exploit. Different question entirely.
- It does **not** assess reachability or exploitability. An EOL library that is never loaded at runtime is a different risk from one in your auth path, and `pld-watch` cannot tell them apart.
- It does **not** read your own source code, compiled binaries, container image layers, lockfile transitive dependencies, or anything outside the files listed above. Transitive dependency trees are a real gap.
- It only knows the frameworks in its internal map, so it will silently miss things it does not recognise. Components it cannot match come back as `"status": "unknown"` — treat that as *not checked*, not as *fine*.
- Its data comes from `endoflife.date`, a community-maintained public catalogue. It is useful and widely used. It is not an authoritative legal or vendor source, and it can be wrong or out of date. For anything load-bearing, confirm against the upstream

vendor’s own lifecycle page.

**In this kit, pld-watch output is treated as one dated input to a record you maintain — never as a conclusion.** File 03 shows exactly how to file it.

**Primary sources used**

Every legal statement in this kit cites one of these. If a statement has no citation, treat it as our engineering opinion, not as law.

- **Directive (EU) 2024/2853** — full text, EUR-Lex:  
<https://eur-lex.europa.eu/eli/dir/2024/2853/oj>
- **Consolidated text** (incorporates the corrigendum), EUR-Lex: <https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX:02024L2853-20241118>
- **Corrigendum 2026/90364** of 7 May 2026 (amends Article 2(1)), EUR-Lex:  
[https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=OJ:L\\_202690364](https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=OJ:L_202690364)
- **Council Directive 85/374/EEC** (the repealed 1985 directive), EUR-Lex:  
<https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX:31985L0374>

Citations in this kit are to **Article** numbers (binding operative text) and **Recital** numbers (numbered paragraphs in the preamble). Recitals are not themselves binding rules, but courts use them to interpret the articles, and for software teams they contain most of the worked examples. Where we rely on a recital we say so.

**A note on the date, because it moved**

You will see both **8 December 2026** and **9 December 2026** in circulation. Both are correct, for different sentences. File 01 explains it precisely. The practical answer for planning is unchanged: **products placed on the EU market from 9 December 2026 onward fall under the new directive.**

**Files in this kit**

| File                | What it gives you                                                     |
|---------------------|-----------------------------------------------------------------------|
| 00-START-HERE.md    | This file. Orientation and limits.                                    |
| 01-AM-I-IN-SCOPE.md | A decision tree to frame whether the directive plausibly reaches you. |

| File                                 | What it gives you                                          |
|--------------------------------------|------------------------------------------------------------|
| 02-WHAT-CHANGES-FROM-1985.md         | Old directive vs new, for engineering and product leaders. |
| 03-UPDATE-SUPPORTABILITY-EVIDENCE.md | The practical core: how to build the record.               |
| 04-SUPPORT-POLICY-TEMPLATES.md       | Four copy-paste templates.                                 |
| 05-BOARD-BRIEF.md                    | Two-page brief for a board or insurer.                     |

## How to keep this current

The text of the directive is stable. What will change is national transposition and, from 2027 onward, case law. Two things are worth a calendar reminder:

- **National transposition law** for each Member State you sell into. Set a review for the first quarter after you start selling there.
- **The Commission judgment database.** Article 19 requires Member States to publish final appellate and highest-instance judgments under this directive, and requires the Commission to set up and maintain a publicly available database of them. Early judgments are where the practical meaning of “necessary to maintain safety” will be settled. [VERIFY: the live URL of the Commission's Article 19 judgment database. As of 14 September 2026 we could not confirm that it is published and reachable. Check the European Commission's product liability pages on [europa.eu](https://european-council.europa.eu/media/1000000000/1/related_content/eu-product-liability).]

---

*Toledo Technologies LLC. Provided as-is, without warranty. Not legal advice.*

# 01 — Am I in scope?

---

**A decision tree for software teams. Directive (EU) 2024/2853.**

---

**Not legal advice.** This file helps you *frame* the scope question and assemble the facts a lawyer will ask for. It cannot answer the question. Scope depends on your facts and on the national law that transposes the directive in each Member State. Nothing here is a compliance determination. Where a branch is genuinely unsettled we mark it [VERIFY: ...] rather than guess.

---

## **Before you start: the directive does not work like a compliance regime**

There is no registration, no notification, no certification, no audit and no regulator that declares you in or out of scope. The directive creates a **private right to compensation** that a natural person can bring in a national court (Article 5(1)). “Scope” therefore means: *if someone were injured and sued, could this directive be the basis of their claim against us?*

That difference matters for how you use this file. You are not filling in a form for an authority. You are working out your own exposure.

---

## **Question 0 — The date. What does the new regime attach to?**

The directive attaches to a **product**, at the moment it is **placed on the market or put into service** — not to your company, not to a calendar year, and not to when the damage happens.

“This Directive shall apply to products placed on the market or put into service after 8 December 2026.” — Article 2(1), **as corrected by Corrigendum 2026/90364 of 7 May 2026**

That corrigendum is easy to miss and a lot of published commentary predates it. The original Official Journal text of Article 2(1) read “after 9 December 2026”; the corrigendum replaced it with “after 8 December 2026”. The consolidated text on EUR-

Lex carries the corrected wording, flagged ▼c1.

The other dates were **not** changed by the corrigendum and still read 9 December 2026:

| Provision                                                                          | Date            | Article                |
|------------------------------------------------------------------------------------|-----------------|------------------------|
| New directive applies to products placed on market / put into service <b>after</b> | 8 December 2026 | Art. 2(1) as corrected |
| Directive 85/374/EEC repealed with effect from                                     | 9 December 2026 | Art. 21                |
| 85/374/EEC continues to apply to products placed on market <b>before</b> that date | —               | Art. 21                |
| Member State transposition deadline                                                | 9 December 2026 | Art. 22(1)             |
| First Commission evaluation                                                        | 9 December 2030 | Art. 20                |

Read together, the intent is clear and the planning answer is simple: **products you place on the EU market from 9 December 2026 onward fall under the new directive; products placed before then stay under the 1985 regime.** The corrigendum removes a one-day gap in the original drafting. Do not build anything around the 8th-versus-9th distinction.

[VERIFY: if your product ships on or within days of 8-9 December 2026 and the exact boundary day could matter to a claim, confirm the transposed national wording in the relevant Member State – transposition statutes may phrase the cut-over differently. This is a lawyer question, not an engineering one.]

**Software-specific trap.** For software, “placed on the market” is not a one-time event you can point at for the lifetime of the product. Two mechanisms keep resetting it:

- A **substantial modification** made after placing on the market makes it, in effect, a new product; where that modification is made by software update or upgrade, or by the continuous learning of an AI system, the modified product is considered made available on the market at the moment the modification is actually made (Recital 40; definition at Article 4(18); modifier treated as manufacturer at Article 8(2)).
- Each new version you ship is its own placing on the market. [VERIFY: whether each

ordinary release of a SaaS product constitutes a fresh "placing on the market" such that a long-lived pre-2026 SaaS product is progressively brought under the new regime. The directive does not say so expressly. This is genuinely unsettled and is one of the most commercially significant open questions for SaaS. Ask a lawyer; do not assume the pre-2026 date protects a continuously deployed service.]

The practical takeaway for a SaaS team: **do not assume that because your service launched in 2019 you are outside the new directive forever.**

---

### Question 1 — Is what you ship a “product”?

“‘product’ means all movables, even if integrated into, or inter-connected with, another movable or an immovable; it includes electricity, digital manufacturing files, raw materials and **software**” — Article 4(1)

Software is named in the operative definition. Recital 13 removes any doubt about delivery model:

“software is a product for the purposes of applying no-fault liability, irrespective of the mode of its supply or usage, and therefore irrespective of whether the software is stored on a device, accessed through a communication network or cloud technologies, or **supplied through a software-as-a-service model.**”

#### In, as products:

| Thing                                                                                                   | Basis                                              |
|---------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Operating systems, firmware, applications, computer programs                                            | Recital 13                                         |
| SaaS                                                                                                    | Recital 13                                         |
| AI systems; AI system providers are treated as manufacturers                                            | Recital 13 (referencing Regulation (EU) 2024/1689) |
| Digital manufacturing files (CAD/3D-print templates with the functional information to drive machinery) | Art. 4(1), Art. 4(2), Recital 16                   |
| Firmware embedded in a device you make                                                                  | Art. 4(1)                                          |

| Thing | Basis |
|-------|-------|
|-------|-------|

### Out, or treated differently:

| Thing                                                                           | Basis      | Caution                                                                                                           |
|---------------------------------------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Information</b> as such — the content of digital files, media files, e-books | Recital 13 | The boundary between “information” and “software” is not defined anywhere in the operative text                   |
| <b>The mere source code of software</b>                                         | Recital 13 | Says <i>mere source code</i> . Shipping a running program is a product; publishing source is treated differently  |
| <b>Services as such</b>                                                         | Recital 17 | But see “related services” below — a digital service that is part of a product is pulled in as a <b>component</b> |
| Damage from nuclear accidents covered by ratified international conventions     | Art. 2(3)  | Irrelevant to most readers                                                                                        |

[VERIFY: where the line falls between “the mere source code of software” (not a product, Recital 13) and software you distribute in source form that a customer compiles and runs commercially. The recital does not resolve this and we found no operative text that does. If you ship source-form commercial software, this is a specific question for your lawyer.]

### Related services — the branch teams miss

“‘related service’ means a digital service that is integrated into, or inter-connected with, a product in such a way that its absence would prevent the product from performing one or more of its functions.” — Article 4(3)

A related service is a **component** of the product it serves (Article 4(4)), so it comes inside the regime even though services as such do not. Recital 17 gives the examples: continuous traffic data in a navigation system; a health monitoring service using a product’s sensors; a temperature control service for a smart fridge; a voice-assistant service controlling products.

Expressly **not** a related service: **internet access services** (Recital 17). But note the

sting in the same recital — “a product that relies on internet access services and fails to maintain safety in the event of a loss of connectivity could be found to be defective.”

**Engineering translation:** if your API or cloud backend is the thing without which a customer’s device or product loses a function, you are probably supplying a component of their product, not merely a service to them.

---

## Question 2 — Which role do you play?

The directive lists the liable **economic operators** in Article 8(1). Work through these in order.

### 2a. Are you a manufacturer?

“‘manufacturer’ means any natural or legal person who: (a) develops, manufactures or produces a product; (b) has a product designed or manufactured, or who, by putting their name, trademark or other distinguishing features on that product, presents themselves as its manufacturer; or (c) develops, manufactures or produces a product for their own use” — Article 4(10)

For a software team, **(a) is the usual answer and it is broad: you develop the software, so you are its manufacturer.** Recital 13 confirms: “A developer or producer of software, including AI system providers ... should be treated as a manufacturer.”

Two traps in that definition:

- **(b) white-labelling.** If you put your name or mark on software someone else built, you are a manufacturer of it. Rebadged, OEM’d and white-labelled products land here.
- **(c) for their own use.** A product you develop and produce for your own use can still make you a manufacturer. [VERIFY: how Article 4(10)(c) interacts with internal-only tooling that is never supplied to anyone. Article 2(1) scopes the directive to products “placed on the market or put into service”, and “putting into service” (Art. 4(9)) means first use in the Union in the course of a commercial activity where the product was not placed on the market first. Whether purely internal software is “put into service” in that sense is not resolved in the text. Ask a lawyer if you run internally-built software in a safety-relevant path.]

## 2b. Are you a component manufacturer?

“the manufacturer of a defective component, where that component was integrated into, or inter-connected with, a product within the manufacturer’s control and caused that product to be defective” — Article 8(1)(b)

If you sell an SDK, a library, an API, an embedded module or a model that gets built into someone else’s product, this is you. The claimant can go after both you and the product manufacturer, and liability is **joint and several** (Article 12(1)) — the injured person can recover the whole amount from either.

There is one relief valve written specifically for small software vendors:

A manufacturer that integrates software as a component shall have **no right of recourse** against the software component manufacturer where that software manufacturer was a **microenterprise or small enterprise** at the time of placing on the market, **and** the two contractually agreed to waive that right. — Article 12(2), size defined by reference to Commission Recommendation 2003/361/EC; see also Recital 54

Read that carefully, because it is narrower than it first looks:

- It removes the **integrator’s recourse against you**. It does **not** remove your liability to the injured person. Article 15 forbids limiting or excluding liability towards the injured person by contract, full stop.
- It requires **an actual contract term**. It is not automatic.
- It requires you to have been micro or small **at the time of placing on the market**, assessed together with partner and linked enterprises.

If you are a small software vendor selling components to larger manufacturers, **this clause is worth raising with your lawyer in your next contract cycle**. It is one of very few places the directive gives you something to negotiate for.

## 2c. Are you an importer, authorised representative or fulfilment provider?

These matter when the manufacturer is **established outside the Union** (Article 8(1)(c)):

- **Importer** — “any natural or legal person who places a product from a third country on the Union market” (Art. 4(12)). If you are a non-EU software vendor and an EU entity puts your product on the EU market, that entity is an importer and is liable alongside you.
- **Authorised representative** — a person established in the Union with a written

mandate from the manufacturer for specified tasks (Art. 4(11)).

- **Fulfilment service provider** — liable only **where there is no importer or authorised representative established in the Union** (Art. 8(1)(c)(iii)). Defined by warehousing/packaging/addressing/dispatching (Art. 4(13)), so for pure software this will rarely bite.

[VERIFY: whether a non-EU SaaS vendor selling directly to EU customers with no EU entity has an "importer" at all, and what that means for where a claim lands. The importer definition is framed around placing a product from a third country on the Union market, which maps awkwardly onto a service delivered over the network. This is a significant open question for non-EU SaaS companies and needs a lawyer.]

## 2d. Are you a distributor or a reseller?

Where an economic operator from Article 8(1) **established in the Union** cannot be identified, each distributor is liable where the injured person asks them to identify such an operator (or their own supplier) and the distributor **fails to do so within one month**. — Article 8(3)

Distributor liability is a fallback, and the escape is procedural and fast: name an EU-established operator within **one month** of the request.

**Operational consequence, and it is a cheap one:** if you resell or distribute other people's software into the EU, keep a record that lets you answer "who made this, and which EU-established entity is on the hook" within a month. That is a supplier register, not a legal project. File 04 includes one.

The same fallback applies to **online platforms** that let consumers conclude distance contracts with traders, where the Article 6(3) DSA conditions are met (Article 8(4), Recital 38).

## 2e. Did you substantially modify someone else's product?

"Any natural or legal person that substantially modifies a product outside the manufacturer's control and thereafter makes it available on the market or puts it into service shall be considered to be a manufacturer of that product." — Article 8(2)

"substantial modification" means a modification of a product after it has been placed on the market or put into service: (a) that is considered substantial under relevant Union or national rules on product safety; or (b) where no such threshold exists, that: (i) changes the product's original

performance, purpose or type, without that change having been foreseen in the manufacturer's initial risk assessment; and (ii) changes the nature of the hazard, creates a new hazard or increases the level of risk." — Article 4(18)

Note limb (b) is **cumulative** — both (i) and (ii) must hold.

This is the **systems integrator and platform-engineering branch**, and it is the one most likely to surprise a team that thinks of itself as a customer rather than a manufacturer. If you take a vendor's product, change it beyond what its maker foresaw, and then supply the result onward, you may have become its manufacturer.

There is a defence available specifically to a modifier: they are not liable if they prove the defectiveness relates to **a part of the product not affected by the modification** (Article 11(1)(g)).

Recital 39 also confirms that economic operators carrying out **repairs or other operations that do not involve substantial modifications** are not liable under this directive.

[VERIFY: whether routine fine-tuning, RAG grounding, prompt-layer wrapping or system-prompt configuration of a third-party foundation model amounts to a "substantial modification" under Art. 4(18)(b), making the integrator a manufacturer of the modified AI system under Art. 8(2). Recital 40 expressly brings "the continuous learning of an AI system" into the substantial modification concept, but does not address deployer-side adaptation. We could find no operative text or Commission guidance resolving this. For anyone building on third-party models this is the single most important open question in the whole directive, and it needs a lawyer.]

---

### Question 3 — Do you place it on the EU market?

"‘making available on the market’ means any supply of a product for distribution, consumption or use on the Union market **in the course of a commercial activity, whether in return for payment or free of charge**" — Article 4(7)

"‘placing on the market’ means the first making available of a product on the Union market" — Article 4(8)

"‘putting into service’ means the first use of a product in the Union in the course of a commercial activity, whether in return for payment or free of charge, in circumstances in which that product has not been placed on the

| market prior to its first use” — Article 4(9)

Three things engineering leaders repeatedly get wrong here:

1. **“Free of charge” does not take you out.** It is in both definitions. Recital 26 adds sponsorship-funded and publicly-funded examples. A free tier, a free developer edition, a free mobile app — all still supplied in the course of a commercial activity if your business is commercial.
2. **The test is commercial activity, not payment.** Recital 14 goes further for software specifically: supply “in exchange for a price, **or for personal data used other than exclusively for improving the security, compatibility or interoperability of the software**” is supply in the course of a commercial activity. Monetising user data instead of charging money does not help you.
3. **It is about the EU market, not your address.** A US company with EU users is supplying the Union market.

[VERIFY: what counts as supplying "the Union market" for a globally-available web application with no EU establishment, no EU targeting and incidental EU users. The directive does not contain a targeting test comparable to the one in consumer jurisdiction rules. This needs a lawyer if it describes you.]

---

## Question 4 — The free and open-source software carve-out

This is the branch most often stated wrongly, including by people who should know better. The operative text is short:

| “This Directive does not apply to free and open-source software that is **developed or supplied outside the course of a commercial activity.**” — Article 2(2)

**The carve-out is about commercial activity, not about the licence.** An MIT or GPL licence does not put you outside the directive. Supplying that software in the course of a commercial activity does put you inside it.

Recital 14 gives the detail, and it is worth reading closely:

- FOSS developed or supplied outside the course of a commercial activity is out, “since products so developed or supplied are **by definition not placed on the market.**”
- “Developing or contributing to such software should not be understood as making

it available on the market.”

- **“Providing such software on open repositories should not be considered as making it available on the market, unless that occurs in the course of a commercial activity.”**
- “In principle, the supply of free and open-source software by **non-profit organisations** should not be considered as taking place in a business-related context, unless such supply occurs in the course of a commercial activity.”
- “However, where software is supplied **in exchange for a price, or for personal data used other than exclusively for improving the security, compatibility or interoperability of the software**, and is therefore supplied in the course of a commercial activity, this Directive should apply.”

### What this means in practice

| Situation                                                                        | Likely position                                                        | Caution                                         |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------|
| You publish a library on GitHub as an individual, unpaid, no commercial offering | Outside — Art. 2(2), Recital 14                                        | Applies to the FOSS supply itself               |
| A non-profit foundation distributes a project, no commercial supply              | “In principle” outside — Recital 14                                    | Recital says “in principle”; it is not absolute |
| You sell a paid support or hosted tier of your own open-source project           | The commercial supply is <b>in the course of a commercial activity</b> | Open-core is the classic hard case              |
| You open-source a component that is also part of your commercial product         | The commercial product is in scope regardless of the licence           | The licence does not shield the product         |
| You are an employee contributing to FOSS as part of your paid job                | [VERIFY] — see below                                                   | Genuinely unsettled                             |

[VERIFY: whether an employee contributing to an open-source project during paid working hours, where the employer benefits commercially from the project, is supplying "outside the course of a commercial activity" under Art. 2(2). Recital 14 says developing or contributing is not making available on the market, which points one way; but "outside the course of a commercial activity" points another. We could not resolve this from the text. If your company's commercial position depends on an open-source project you sponsor, ask a lawyer.]

[VERIFY: how Art. 2(2) applies to dual-licensed and open-core models where the same codebase is supplied both non-commercially and commercially. The text does not address it directly.]

### The most important consequence for consumers of FOSS

Where FOSS supplied outside the course of a commercial activity is **subsequently integrated by a manufacturer as a component into a product in the course of a commercial activity** and thereby placed on the market, **that manufacturer** can be held liable for damage caused by the defectiveness of such software — **but not the manufacturer of the software**, because it would not have fulfilled the conditions of placing a product on the market. — Recital 15

Read that twice if you build on open source, because it is the line that makes this whole kit worth doing:

**The liability for a defect in the non-commercial open-source component you shipped lands on you, not on the upstream maintainer.**

That is the honest, unglamorous reason a component inventory matters. The unpaid volunteer who wrote the library you depend on is carved out. You, who put it in a commercial product, are not. And if that component is past end of life, nobody upstream is going to ship you a security fix. That is precisely the fact `pld-watch` was built to surface, and precisely why File 03 is the heart of this kit.

---

### Question 5 — Could anyone actually claim against you?

Being in scope is not the same as being exposed. A claim needs recoverable damage. The directive covers **only** these (Article 6(1)):

- **(a)** death or personal injury, including medically recognised damage to psychological health;
- **(b)** damage to, or destruction of, any property, **except**: (i) the defective product itself; (ii) a product damaged by a defective component the manufacturer integrated or that was within its control; (iii) **property used exclusively for professional purposes**;
- **(c) destruction or corruption of data that are not used for professional purposes.**

Recoverable to **natural persons** only (Article 5(1)). And note what is **excluded** — Recital

24: “Types of damage other than those provided for in this Directive, such as **pure economic loss, privacy infringements or discrimination**, should not by themselves trigger liability under this Directive.”

**So, bluntly:** a pure B2B SaaS tool used exclusively by businesses on business-owned equipment, whose realistic failure mode is downtime or financial loss, has materially less PLD exposure than the internet is telling you. Downtime is pure economic loss. Business property is excluded. A data breach is a GDPR matter, and Recital 20 expressly distinguishes destruction/corruption of data from data leaks and data protection infringements.

Where software exposure is real:

- Software that can cause **physical harm** — medical, industrial, automotive, robotics, building control, anything driving hardware.
- Software handling **consumers’ personal (non-professional) data** where a defect could **destroy or corrupt** it — consumer backup, photo storage, note apps, password managers, consumer cloud sync.
- **Consumer** devices and apps generally, where the user is a natural person and the property is theirs.
- Components inside any of the above, including AI components.

Note the mixed-use rule: property used for both private and professional purposes **is** covered (Recital 25), only *exclusively* professional property is out. But for **data** the rule is stricter — data used for professional purposes, “even if not exclusively so”, is excluded (Recital 22). Those two rules differ. That is deliberate, and it matters for a device or app used for both work and home.

---

## Your scope card

Fill this in, date it, and keep it with the evidence file from File 03.

|                                                        |                        |
|--------------------------------------------------------|------------------------|
| PRODUCT SCOPE CARD                                     | Date: _____            |
| Product / service: _____                               |                        |
| Owner: _____                                           |                        |
| Q0 First placed on EU market / put into service: _____ |                        |
| On or after 9 Dec 2026?                                | [ ] Y [ ] N            |
| Substantially modified since?                          | [ ] Y [ ] N [ ] Unsure |
| If continuously deployed SaaS, flagged for legal?      | [ ] Y                  |

Q1 Is it software / a related service / a digital manufacturing file? ☐ Y ☐ N

Which: ☐ standalone software ☐ SaaS ☐ firmware  
☐ AI system ☐ component/SDK/API  
☐ related service to someone else's product

Q2 Our role(s):

- ☐ Manufacturer (Art. 4(10)(a) - we develop it)
- ☐ Manufacturer by name/mark (Art. 4(10)(b))
- ☐ Component manufacturer (Art. 8(1)(b))
- ☐ Importer (Art. 4(12))
- ☐ Authorised representative (Art. 4(11))
- ☐ Fulfilment service provider (Art. 4(13))
- ☐ Distributor (Art. 4(14))
- ☐ Substantial modifier (Art. 8(2))

If micro/small enterprise supplying software components:

Art. 12(2) recourse waiver raised with legal? ☐ Y ☐ N

Q3 Supplied on the EU market in the course of a commercial activity (incl. free tiers)? ☐ Y ☐ N

Q4 Any part supplied as FOSS outside commercial activity?

☐ N/A ☐ Yes - describe: \_\_\_\_\_

FOSS components we ship inside our commercial product:  
(liability for these is ours - Recital 15)

Count from pld-watch / SBOM: \_\_\_\_\_

Q5 Realistic damage types if our product failed badly:

- ☐ Death or personal injury (incl. psychological)
- ☐ Damage to non-professional property
- ☐ Destruction/corruption of non-professional data
- ☐ None of the above - only downtime / financial loss  
(= pure economic loss, not covered: Recital 24)

PROVISIONAL VIEW: ☐ plausibly in scope

☐ plausibly out of scope

☐ cannot tell without advice

Questions for our lawyer (list every [VERIFY] that applied):

1. \_\_\_\_\_

2. \_\_\_\_\_

3. \_\_\_\_\_

Completed by: \_\_\_\_\_ Reviewed by: \_\_\_\_\_

## When to stop and get a lawyer

Stop and take advice — do not reason your way past these:

1. Your software could contribute to **physical injury**. That is the highest- exposure branch and it is not a self-assessment matter.
2. You build on **third-party foundation models** and adapt them. See the substantial-modification [VERIFY] above.
3. You are **non-EU with EU users and no EU entity**.
4. Your commercial model is **open-core, dual-licensed, or a sponsored open source project**.
5. You are a **small software component vendor** and want the Article 12(2) recourse waiver in your contracts.
6. You are about to **publish a support policy** making public commitments about security updates. Everything in File 04 needs legal review before it goes out.
7. Any answer above was “cannot tell”.

---

*Toledo Technologies LLC. Not legal advice. Sources: Directive (EU) 2024/2853, <https://eur-lex.europa.eu/eli/dir/2024/2853/oj> ; consolidated text <https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX:02024L2853-20241118> ; Corrigendum 2026/90364 [https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=OJ:L\\_202690364](https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=OJ:L_202690364)*

## 02 — What changes from 1985

---

### Directive 85/374/EEC vs Directive (EU) 2024/2853, for engineering and product leaders.

---

**Not legal advice.** This is a plain-English comparison of two legal texts, written by engineers for engineers. It is not legal advice, not a compliance determination, and not a substitute for reading the directive or taking advice. Both directives are linked at the end. Where we could not verify something we say so with a [VERIFY: ...] marker instead of guessing.

---

### The one-paragraph version

The 1985 directive already imposed **no-fault (strict) liability**: if your product was defective and it hurt someone, you paid, whether or not you were careless. That has not changed and was never the news. What changed in 2024 is **what counts as a product** (software is now named in the text), **how long your responsibility runs** (control now follows the update channel, not the shipping date), **how hard it is for a claimant to prove their case** (new disclosure duty and new presumptions), and **what you can be made to pay for** (data loss is in, the €500 property threshold is gone).

For a software team, the change with teeth is this: **you can no longer rely on “the defect appeared after we shipped it” for a defect that consists in the absence of a security update you could have supplied.**

---

### Side by side

|                        | 85/374/EEC (1985)                                                                                          | (EU) 2024/2853 (2024)                                                                                                                                                              |
|------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Is software a product? | Not mentioned. “all movables ... includes electricity” (Art. 2). Whether software qualified was argued for | Yes, expressly. “includes electricity, digital manufacturing files, raw materials and <b>software</b> ” (Art. 4(1)); SaaS and AI covered regardless of delivery model (Recital 13) |

30 years

|                                      |                                                                                                                                    |                                                                                                                                                                                                                                     |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Services</b>                      | Out of scope                                                                                                                       | Services as such still out, <b>but</b> a “related service” without which the product loses a function is a <b>component</b> and comes in (Art. 4(3), 4(4), Recital 17)                                                              |
| <b>Who is liable</b>                 | Producer, component producer, own-branding, importer into the Community; supplier as fallback (Art. 3)                             | Manufacturer, component manufacturer, <b>importer, authorised representative, fulfilment service provider</b> ; distributor and certain online platforms as fallback; <b>substantial modifier treated as manufacturer</b> (Art. 8)  |
| <b>When does responsibility end?</b> | At “putting into circulation”. Defence if the defect “came into being afterwards” (Art. 7(b))                                      | Same defence (Art. 11(1)(c)) — <b>but disappplied</b> for related services, software and updates, <b>lack of safety-necessary updates</b> , and substantial modification, where within the manufacturer’s control (Art. 11(2))      |
| <b>Defectiveness test</b>            | Safety a person is entitled to expect, considering presentation, expected use, time of circulation — <b>three factors</b> (Art. 6) | Same core test, <b>nine listed factors</b> including learning ability, interconnection effects, <b>safety-relevant cybersecurity requirements</b> , recalls, and the moment the product left the manufacturer’s control (Art. 7(2)) |
| <b>Burden of proof</b>               | Claimant proves damage, defect, causation. Full stop (Art. 4)                                                                      | Same baseline (Art. 10(1)) <b>plus four routes to a presumption</b> (Art. 10(2)-(4))                                                                                                                                                |
| <b>Disclosure of evidence</b>        | Nothing                                                                                                                            | Court can order the <b>defendant</b> to disclose relevant evidence, including documents created ex novo (Art. 9, Recital 42). Non-disclosure <b>presumes defectiveness</b> (Art. 10(2)(a))                                          |

|                           | 85/374/EEC (1985)                                                                                                    | (EU) 2024/2853 (2024)                                                                                                                                                                                    |
|---------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Recoverable damage</b> | Death, personal injury; property damage for private-use items, <b>with a €500 (500 ECU) lower threshold</b> (Art. 9) | Death, personal injury <b>including medically recognised psychological harm</b> ; property damage <b>with no lower threshold</b> ; <b>destruction or corruption of non-professional data</b> (Art. 6(1)) |
| <b>Financial ceiling</b>  | Member State could cap total liability for identical items at not less than 70 million ECU (Art. 16)                 | <b>No such option.</b> Liability must not be limited or excluded by contract or by national law (Art. 15, Recital 56)                                                                                    |
| <b>Limitation period</b>  | 3 years from awareness of damage, defect, producer identity (Art. 10)                                                | 3 years, same trigger (Art. 16)                                                                                                                                                                          |
| <b>Expiry period</b>      | 10 years from putting into circulation (Art. 11)                                                                     | 10 years from placing on market / putting into service; <b>25 years for latent personal injury; restarts on substantial modification</b> (Art. 17, Art. 17(1)(b))                                        |
| <b>Harmonisation</b>      | Minimum-ish, with options for Member States                                                                          | <b>Maximum harmonisation</b> — no more stringent or less stringent national provisions (Art. 3)                                                                                                          |
| <b>FOSS</b>               | Nothing                                                                                                              | Express carve-out for FOSS developed or supplied <b>outside the course of a commercial activity</b> (Art. 2(2), Recitals 14-15)                                                                          |
| <b>Transparency</b>       | None                                                                                                                 | Member States publish final appellate judgments; Commission maintains a public database (Art. 19)                                                                                                        |

# The five changes that matter to an engineering team

## 1. Software is a product, and the delivery model does not save you

The old fight was whether software was a “movable”. That fight is over. Article 4(1) names software. Recital 13 covers “operating systems, firmware, computer programs, applications or AI systems”, and says the answer does not change whether the software is “stored on a device, accessed through a communication network or cloud technologies, or supplied through a software-as-a-service model.”

**What it does not cover:** “Information is not ... to be considered a product”, so “the content of digital files, such as media files or e-books or the mere source code of software” is out (Recital 13). The line between information and software is not defined in the operative text.

**Engineering translation:** the architecture diagram does not change your exposure. Shipping a binary, running a cloud service, and embedding firmware all land in the same place.

## 2. The update channel is now a control channel — and control is the hinge

This is the structural change. Under 85/374/EEC, “putting into circulation” was effectively a finish line. The 2024 directive replaces the finish line with a question: **do you still have control?**

“‘manufacturer’s control’ means that: ... (b) the manufacturer of a product **has the ability to supply software updates or upgrades, themselves or via a third party.**” — Article 4(5)(b)

Note what that says. Not *does supply*. Not *promised to supply*. **Has the ability to supply**. Recital 19 restates it: once placed on the market, a product “should be considered to remain within the manufacturer’s control where the manufacturer retains the ability to supply software updates or upgrades itself or via a third party.”

Control also covers components and updates that you **authorise or consent to** from a third party (Art. 4(5)(a)(i)). Recital 18 draws the boundary usefully: you are **not** taken to have consented “merely by providing for the technical possibility of integration or inter-connection or by recommending certain brands or by not prohibiting potential related services or components.” So a plugin API alone is not consent; presenting a third-party component as part of your product is.

Control then does two jobs:

- It is a **factor in defectiveness** — Article 7(2)(e) directs the court to the moment the product was placed on the market “or, where the manufacturer retains control over the product after that moment, the moment in time when the product left the control of the manufacturer.”
- It **switches off the later-defect defence** — Article 11(2), below.

### 3. The “later defect” defence, narrowed

Old law, Article 7(b) of 85/374/EEC: the producer is not liable if it proves “that ... it is probable that the defect which caused the damage did not exist at the time when the product was put into circulation by him or that this defect came into being afterwards.”

New law, Article 11(1)(c) of 2024/2853, substantially the same wording. Then:

“By way of derogation from paragraph 1, point (c), an economic operator shall not be exempted from liability where the defectiveness of a product is due to any of the following, **provided that it is within the manufacturer’s control**: (a) a related service; (b) software, including software updates or upgrades; (c) **a lack of software updates or upgrades necessary to maintain safety**; (d) a substantial modification of the product.” — Article 11(2)

Point (c) is the one that should be on your wall. Combined with Article 4(5)(b), the chain runs:

You retain the ability to ship an update  
 -> the product is within your control (Art. 4(5)(b))  
 -> and the defect consists in the lack of an update  
     necessary to maintain safety  
 -> then “it happened after we shipped” is not available  
     to you (Art. 11(2)(c))

**Now the limits, because this is where commentary overreaches.**

“Such liability **should not apply where the supply or installation of such software is beyond the manufacturer’s control**, for example where the owner of the product does not install an update or upgrade supplied for the purpose of ensuring or maintaining the level of safety of the product. **This Directive does not impose any obligation to provide updates or upgrades for a product.**” — Recital 51 (final two sentences)

Three honest points follow:

1. **There is no patching mandate in this directive.** Anyone telling you the PLD requires you to patch is overselling. (Other EU law may impose update obligations on you — the Cyber Resilience Act is the obvious candidate — but that is a different instrument with its own scope and dates, and it is outside this kit.)
2. **The trigger is a defect, not a missing patch.** Article 11(2)(c) bites where the *defectiveness* consists in the lack of updates *necessary to maintain safety*. An unpatched component that does not make the product less safe than the public at large is entitled to expect is not, by itself, a defect.
3. **If the user does not install what you supplied, that is expressly outside your control** (Recital 51). And Article 13(2) allows liability to be reduced or disallowed where the injured person's own fault contributed — Recital 55 gives the example of an injured person who “negligently failed to install updates or upgrades provided by the economic operator”.

So the defensible engineering position is not “patch everything forever”. It is: **be able to ship a security update for what you ship; actually ship them where they are needed for safety; tell people clearly when you stop; and write down what you decided and why.** That is exactly what File 03 builds.

One more asymmetry worth knowing, from Article 13(1): where damage is caused **both** by your product's defectiveness **and** by a third party's act or omission, your liability is **not** reduced. Recital 55 names the case explicitly — “a third party exploiting a cybersecurity vulnerability of a product”. *Someone attacked us* is not a defence.

#### 4. Disclosure and presumptions — the procedural squeeze

Under the old directive the claimant proved everything and had no route into your internal records. Two new mechanisms change the dynamics.

**Disclosure (Article 9).** Where a claimant “has presented facts and evidence sufficient to support the plausibility of the claim”, the court requires the defendant to disclose relevant evidence at its disposal. Guardrails: disclosure is limited to what is “necessary and proportionate” (Art. 9(3)); courts weigh confidentiality and **trade secrets** (Art. 9(4)) and can order specific measures to protect them during and after proceedings (Art. 9(5)). It cuts both ways — a defendant can seek disclosure from the claimant (Art. 9(2)).

Two details engineers should notice:

- Recital 42: disclosable evidence “includes documents that have to be **created ex novo by the defendant by compiling or classifying the available evidence.**” You may be made to *assemble* a record, not merely hand over files that already

exist.

- Article 9(6): the court can require evidence to be presented “in an easily accessible and easily understandable manner” where proportionate.

**Presumptions (Article 10).** Defectiveness is presumed where:

- **(2)(a)** the defendant **fails to disclose** relevant evidence under Art. 9(1) — the sanction for stonewalling is losing the point;
- **(2)(b)** the claimant shows the product does not comply with **mandatory product safety requirements** in Union or national law intended to protect against that risk;
- **(2)(c)** the claimant shows the damage was caused by an **obvious malfunction** in reasonably foreseeable use.

Causation is presumed where the product is established as defective and the damage is “of a kind typically consistent with the defect in question” (Art. 10(3)).

And the complexity valve (Art. 10(4)): the court **shall** presume defectiveness or causation or both where, despite disclosure, the claimant faces **excessive difficulties** proving them “in particular due to technical or scientific complexity”, **and** shows it is **likely** that the product is defective or that causation exists. Recital 48 names machine learning and “a link that, in order to be proven, would require the claimant to explain the inner workings of an AI system” as examples, and says the claimant need not prove the difficulty, only argue it.

All presumptions are **rebuttable** (Art. 10(5)). The defendant can contest every element, including whether excessive difficulty exists.

**Engineering translation:** the quality of your records is now part of your defence in a way it was not before. A team that can produce a clear, dated, contemporaneous account of what it shipped, what it knew and what it decided is in a materially different position from a team reconstructing it under a court order two years later. That is not a legal opinion; it is a practical observation about evidence. But it is the reason this kit exists.

## 5. Money and time — what you can be made to pay, and for how long

**The €500 property threshold is gone.** Article 9(b) of 85/374/EEC set a 500 ECU lower threshold for property damage. There is no equivalent in Article 6 of the new directive. Small property claims are now in.

**Data loss is in.** Article 6(1)(c): “destruction or corruption of data that are not used for

professional purposes.” Recital 20 explains — deleted digital files, “including the cost of recovering or restoring those data”.

The limits on that are important and often dropped in summaries:

- **Only non-professional data.** And the exclusion is wider than for property: data used for professional purposes “**even if not exclusively so**” is out (Recital 22). Mixed-use *property* is covered; mixed-use *data* is not.
- **No automatic loss.** “Destruction or corruption of data does not automatically result in a material loss if the victim is able to retrieve the data at no cost, such as where a back-up of the data exists or the data can be downloaded again” (Recital 20). A working restore path is a genuine mitigator.
- **Not a breach regime.** “Destruction or corruption of data is distinct from data leaks or breaches of data protection rules” (Recital 20). GDPR handles that; this does not.

**Psychological harm is expressly in** — “medically recognised damage to psychological health” (Art. 6(1)(a)), meaning medically recognised and certified harm affecting general health and possibly requiring treatment (Recital 21).

**Still out:** the defective product itself; property used exclusively for professional purposes; and per Recital 24, pure economic loss, privacy infringements and discrimination do not by themselves trigger liability here.

**No caps, no contracting out.** Article 15 forbids limiting or excluding liability towards the injured person by contract **or by national law**. The old Article 16 national ceiling option is gone. Your EULA’s limitation of liability clause does not touch a PLD claim.

**Time.** Limitation: 3 years from when the injured person knew or should have known of damage, defectiveness and the liable operator’s identity (Art. 16) — unchanged in substance. Expiry: **10 years** from placing on the market or putting into service (Art. 17(1)); **25 years** where personal injury was latent and the claimant could not sue within 10 (Art. 17(2)); and for a **substantially modified** product the 10 years runs afresh from when the modified product was made available (Art. 17(1)(b)).

So a product placed on the market on 9 December 2026 carries a claim window running to **9 December 2036**, and up to **9 December 2051** for latent personal injury. Recital 58 confirms the sensible corollary: ordinary updates or upgrades that do **not** amount to a substantial modification do **not** restart the clock.

**Records implication:** whatever you keep as evidence of update supportability needs a

retention period measured in **years, not sprints**. See File 03.

---

## What did NOT change

Worth saying plainly, because the change is oversold in places:

- **Strict liability is not new.** 1985 was already no-fault. Recital 2 of the new directive repeats the 1985 rationale nearly word for word.
  - **The claimant still bears the burden.** Article 10(1) keeps damage, defect and causation on the claimant. The presumptions are conditional and rebuttable, not a reversal. Recital 48 says so in terms — the aim is to maintain fair apportionment “while avoiding a reversal of the burden of proof”.
  - **Only natural persons can claim** (Art. 5(1)). This is not a business-to-business damages regime.
  - **A better product later does not prove the earlier one was defective** (Art. 7(3)) — and expressly, that includes shipping updates or upgrades. Recital 35 confirms: “the supply of updates or upgrades for a product should not in itself lead to the conclusion that a previous version of the product is defective.” **Patching is not an admission.** If anyone in your organisation argues against shipping a fix because it looks like an admission, this is the citation that answers them.
  - **The development risk defence survives** (Art. 11(1)(e)) — no liability if the objective state of scientific and technical knowledge, at the time of placing on the market **or during the period the product was within the manufacturer’s control**, was not such that the defectiveness could be discovered. Note the extension to the control period: for software you keep updating, the knowledge yardstick moves with you. Member States may derogate from this defence for specific product categories under Article 18. [VERIFY: whether any Member State relevant to you has maintained or introduced an Art. 18 derogation from the development risk defence. Art. 18(1) required notification of maintained measures to the Commission by 9 December 2026. Check the national transposition law in each Member State you sell into.]
  - **Compliance with legal requirements** remains a defence where compliance is what caused the defectiveness (Art. 11(1)(d)).
  - **Component manufacturers keep their defence** where defectiveness is attributable to the design of the product they went into, or the integrator’s instructions (Art. 11(1)(f)).
-

## What this means for your roadmap

Nothing here requires a compliance programme. It suggests five habits, all of which are ordinary good engineering that now also happens to be evidence:

1. **Know what you ship.** A component inventory you can produce on demand.
2. **Know what can no longer be patched.** EOL tracking. This is the pld-watch question.
3. **Be able to ship a security update** for everything you support — and know which things you cannot.
4. **Say publicly what you support and until when.** A stated support window is how a reasonable expectation gets set, and Article 7(2)(a) puts the presentation of the product, including instructions for use and maintenance, into the defectiveness assessment.
5. **Write down decisions not to patch,** at the time, with reasons.

File 03 turns those five into a concrete afternoon of work.

---

*Toledo Technologies LLC. Not legal advice.*

*Sources: Directive (EU) 2024/2853 — <https://eur-lex.europa.eu/eli/dir/2024/2853/oj> ; consolidated text incorporating Corrigendum 2026/90364 — <https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX:02024L2853-20241118> ; Council Directive 85/374/EEC — <https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX:31985L0374>*

## 03 — Update supportability evidence

---

**How to build and keep a defensible record that you can, and do, supply security updates for what you ship.**

---

**Not legal advice.** This file describes an engineering and record-keeping practice. It is not legal advice, not a certification, and not a compliance determination. Following it does not make you compliant, does not make your product safe, and does not determine whether you are liable. It is designed to make a record that exists, is dated, and is honest — nothing more, and nothing less. Have your legal adviser review anything you publish, and review your retention periods against the liability periods before you set them.

---

### What this is for, in one paragraph

Under Article 11(2)(c), you cannot escape liability by saying the defect arose after you shipped, where the defectiveness is due to **“a lack of software updates or upgrades necessary to maintain safety”** and the product is within your control. Under Article 4(5)(b), the product is within your control simply because you **have the ability** to supply updates. So the useful engineering question is not “are we compliant” — there is nothing to comply with. It is:

**For everything we ship: can we supply a security update for it? Do we? And when we decided not to, did we decide it, or did it just happen?**

A record that answers those three questions is worth building. Not because it wins a case — nobody can promise you that — but because the alternative is reconstructing it from Slack two years later, possibly under an Article 9 disclosure order.

### Why the record, specifically

Three provisions make contemporaneous records matter more than they used to:

- **Article 9** — a court can order you to disclose relevant evidence, and per Recital 42

that includes “documents that have to be created **ex novo** by the defendant by compiling or classifying the available evidence.” You may be ordered to *assemble* the picture. Assembling it now is cheaper.

- **Article 10(2)(a)** — if you fail to disclose when ordered, **defectiveness is presumed**. Being unable to produce records is not a neutral position.
- **Article 7(2)** — the defectiveness assessment weighs “all circumstances”, including presentation and “instructions for its assembly, installation, use and **maintenance**” (7(2)(a)), “relevant product safety requirements, including **safety-relevant cybersecurity requirements**” (7(2)(f)), and the moment the product left your control (7(2)(e)). Your stated support position is part of the factual matrix.

**What a record cannot do.** It cannot make a defective product non-defective. It cannot prove you were not negligent — negligence is not the test; liability is no-fault. It cannot substitute for actually shipping the update. Be clear-eyed about this: you are building a factual record, not a shield.

---

## The five artefacts

| # | Artefact                                  | Answers                                              | Effort to start | Where it lives     |
|---|-------------------------------------------|------------------------------------------------------|-----------------|--------------------|
| 1 | <b>Component inventory</b>                | What is actually in what we ship?                    | 1-2 h           | Repo, versioned    |
| 2 | <b>EOL register</b>                       | What can no longer receive an upstream security fix? | 1 h             | Repo, versioned    |
| 3 | <b>Update policy with support windows</b> | What do we tell people we support, and until when?   | 2-3 h + legal   | Public page        |
| 4 | <b>End-of-support notices</b>             | Did we tell people when we stopped?                  | 30 min each     | Public + archived  |
| 5 | <b>Decision-not-to-update records</b>     | Where we did not patch, why?                         | 15 min each     | Internal, retained |

Templates for 3, 4, 5 and a register format for 2 are in 04-SUPPORT-POLICY-TEMPLATES.md.

Start at 1 and 2. They are the ones only you can build, and the ones that decay fastest if

you leave them.

---

## Artefact 1 — Component inventory

### What good looks like

For each released, supported version of each product, a list of the components it contains, with: name, version, where it came from, licence, whether it is upstream-supported, and who on your side owns it.

If you already produce an **SBOM** (CycloneDX or SPDX) in CI, you are most of the way there. An SBOM is the right primitive — it is machine-readable, it is generated rather than typed, and it can be attached to a release artefact so it is genuinely contemporaneous.

[VERIFY: whether SBOM generation is mandatory for you under other EU law – notably the Cyber Resilience Act, Regulation (EU) 2024/2847 – and on what timeline. Directive (EU) 2024/2853 itself imposes no SBOM requirement. We deliberately do not state CRA obligations or dates in this kit because CRA scope and timing are outside what we verified here. Check the CRA text on EUR-Lex and take advice.]

### Minimum viable version

If you cannot produce an SBOM today, do not let that stop you. A dated CSV per released version beats nothing, and beats an SBOM you keep meaning to set up.

```
component,version,source,type,licence,upstream_supported,eol_date,owner,notes
node,16.20.0,.nvmrc,runtime,MIT,no,2023-09-11,platform-team,pinned for native addon compat
next,12.3.1,package.json,framework,MIT,no,2022-11-21,web-team,upgrade planned Q1
openssl,3.0.13,Dockerfile base,library,Apache-2.0,yes,,platform-team,
libfoo,1.2.0,vendored source,library,BSD-3,unknown,,integrations,vendored 2023, upstream
archived
```

### Rules that make this hold up

1. **Per released version, not per repo HEAD.** The question a claim asks is “what was in the thing the customer had”, not “what is on main today”.
2. **Generate it at build time and attach it to the release.** A generated, timestamped artefact is contemporaneous evidence. A spreadsheet someone typed last week is not.
3. **Record vendored and copied code.** Vendored dependencies, forked libraries and copy-pasted snippets do not appear in any manifest and are exactly where

unmaintained code hides.

4. **Record what you could not determine.** An explicit unknown is honest and useful. A silent gap looks, later, like a record that was curated.
5. **Keep it for the liability period.** See the retention section below.

### The gap you must not paper over

A manifest-derived inventory covers **declared direct dependencies**. It does not by itself cover the **transitive dependency tree**, OS packages in your base image, binaries you ship, or code you vendored years ago. If your inventory is manifest-derived, write that limitation *into the inventory*. A record that overstates its own coverage is worse than one that states its limits, because somebody will eventually check.

---

## Artefact 2 — EOL register, and where `pld-watch` fits

### The narrow question the tool answers

`pld-watch scan` reads the runtimes and frameworks a project **declares** and checks each version against the public **endoflife.date** catalogue. Files it reads: `.nvmmrc`, `.python-version`, `go.mod`, `package.json` (engines plus a built-in map of known frameworks), `requirements.txt` / `requirements-dev.txt`, and `FROM` lines in a Dockerfile.

That is one checkable fact: **is this declared version past its upstream EOL date?** It matters because past EOL means *no upstream security fix is coming*. If a safety-relevant vulnerability appears in it, you cannot ship a patch for it without first migrating off it. That is a real constraint on your ability to maintain safety, and Article 4(5)(b) frames control in exactly those terms — the ability to supply updates.

### Running it

```
curl -fsSL https://raw.githubusercontent.com/ntoledo319/pld-watch/main/pld_watch.py -o pld-watch
chmod +x pld-watch

./pld-watch scan                # human-readable
./pld-watch scan --json         # machine-readable, for CI and for the record
./pld-watch scan /path/to/project
./pld-watch clock               # days until the directive applies
```

Exit codes: 0 nothing past EOL · 1 at least one component past EOL · 2 nothing detectable to check.

Note 2 carefully. **“Nothing detectable” is not “nothing wrong”**. It means the tool found no files it understands. Treat exit code 2 as *unscanned*.

**The --json output, and what each field is worth**

A real run against a small project with a pinned old Node and an old Next.js:

```
{
  "tool": "pld-watch",
  "version": "1.0.0",
  "scanned": "/tmp/pldtest",
  "pld_applies": "2026-12-09",
  "generated_utc": "2026-09-14T20:08:29+00:00",
  "components": [
    {
      "kind": "runtime",
      "name": "Node.js",
      "slug": "nodejs",
      "version": "16.20",
      "source": ".nvmrc",
      "status": "eol",
      "eol_date": "2023-09-11",
      "latest": "26"
    },
    {
      "kind": "framework",
      "name": "next",
      "slug": "nextjs",
      "version": "12.3",
      "source": "package.json",
      "status": "eol",
      "eol_date": "2022-11-21",
      "latest": "16"
    }
  ],
  "eol_count": 2,
  "disclaimer": "Engineering tooling, not legal advice."
}
```

Field by field, and what it is actually evidence of:

| Field         | Meaning           | What it is evidence of                                              |
|---------------|-------------------|---------------------------------------------------------------------|
| generated_utc | When the scan ran | <b>The most useful field for your record.</b><br>It dates the check |
| scanned       | Path scanned      | What was, and was not, in the scan                                  |

| Field                              | Meaning                                                                              | What it is evidence of                            |
|------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------|
| <code>components[].source</code>   | The file the version came from                                                       | Traceability — you can re-derive the finding      |
| <code>components[].status</code>   | <code>eol</code> , <code>soon</code> , <code>supported</code> , <code>unknown</code> | See below. <code>unknown</code> is not supported  |
| <code>components[].eol_date</code> | Upstream EOL date from <code>endoflife.date</code>                                   | The underlying fact, checkable against the vendor |
| <code>components[].latest</code>   | Latest non-EOL release known                                                         | Where a migration would go                        |
| <code>days_to_eol</code>           | Present on non-EOL matches                                                           | Lead time for planning                            |
| <code>eol_count</code>             | Count of <code>eol</code> entries                                                    | A trend line if you keep scans over time          |

status values, precisely as the tool computes them: `eol` if the matched release is flagged end-of-life upstream or its EOL date has passed; `soon` if EOL is within 180 days; `supported` if EOL is further out or none is published; `unknown` if the component or version could not be matched at all.

**unknown means not checked.** It is the single most misread field. Record `unknown` entries in your register as open items, not as clean ones.

### How to file it — the practice

Run it in CI on every release build and **store the JSON as a release artefact**, next to the SBOM. That gives you, per version shipped, a dated machine-readable statement of what was past EOL at the moment of shipping. That is the artefact that is impossible to recreate later.

```
# Example CI step. Adapt to your platform.
# Deliberately does not fail the build: an EOL finding is
# information to be triaged, not an automatic release blocker.
- name: pld-watch EOL scan
  run: |
    curl -fsSL https://raw.githubusercontent.com/ntoledo319/pld-watch/main/pld_watch.py -o
    pld-watch
    chmod +x pld-watch
```

```
./pld-watch scan --json > "pld-watch-${GITHUB_SHA}.json" || true
cat "pld-watch-${GITHUB_SHA}.json"
- name: Archive scan
  uses: actions/upload-artifact@v4
  with:
    name: pld-watch-scan
    path: pld-watch-*.json
    retention-days: 90    # then move to long-term storage; see retention below
```

A deliberate choice worth explaining: **do not fail the build on exit code 1**. An EOL component is a fact to be triaged and recorded, not a defect. Gating releases on it trains people to suppress the signal, and a suppressed signal is worse than no signal — including as evidence.

### What a scan proves, and what it does not

Be precise about this, especially in anything you show a board, a customer or an insurer.

**A pld-watch scan is evidence that:** on a stated date, a stated set of declared components was checked against a public EOL catalogue, and these were past EOL.

### A pld-watch scan is NOT evidence that:

- your product is safe, or not defective — Article 7 decides defectiveness on all the circumstances, and no scanner applies that test;
- you are, or are not, in scope;
- you have no vulnerabilities — **it is not a CVE or SCA scanner**, it does not look at known vulnerabilities at all;
- your dependency tree is clean — it reads declared direct versions, not the transitive graph, not lockfiles, not OS packages, not container image layers, not your own source;
- an EOL component is reachable or exploitable in your system. An EOL parser never invoked at runtime and an EOL TLS library in your auth path both appear as one line of output. They are not the same risk and the tool cannot tell them apart.

**That judgement is yours and it belongs in your register.**

**And on the data source:** endoflife.date is a community-maintained public catalogue. It is widely used and generally good. It is not authoritative, not a vendor source, and not a legal source. For any component where the EOL status is load-bearing in a decision you record, confirm against the vendor's own lifecycle page and cite that in your register.

## Use pld-watch as one input among several

A defensible EOL register draws on more than one tool. pld-watch answers “past upstream EOL”. You will also want, at minimum:

- an **SCA / vulnerability scanner** for known CVEs (different question);
- **SBOM generation** for the full component graph including transitives;
- **base image lifecycle** tracking for your containers;
- **vendor lifecycle pages** for commercial dependencies.

Nothing in this kit suggests pld-watch replaces any of those. It covers one narrow fact that the others often do not surface plainly: *this can no longer be patched at all*.

## The register itself

The JSON is an input. The register is the human layer where judgement is recorded — reachability, decision, owner, date. The tool cannot produce those. Format and a filled example are in File 04.

---

## Artefact 3 — Update policy with stated support windows

### Why a stated window helps you

Article 7(1) frames defectiveness as the safety “a person is entitled to expect”. Article 7(2)(a) puts the product’s presentation — including “instructions for its assembly, installation, use and **maintenance**” — into that assessment. A clear, public, honest statement of what you support and for how long is part of how a reasonable expectation is formed.

Two cautions, stated plainly:

1. **A short support window is not a liability shield.** Article 15 prohibits limiting or excluding liability towards the injured person by contract or by national law. A support policy is not a contractual liability cap and will not function as one. Do not let anyone sell it to you as one.
2. **An overstated window is worse than none.** If you publish “security updates for 5 years” and do not deliver them, you have created a documented gap between what you said and what you did. Publish only what you will actually do. [VERIFY: whether a published support commitment creates separate contractual or consumer-law obligations in your target Member States. It very plausibly does, independently of the PLD. Legal review before publishing.]

## What a usable policy states

- Which products and versions it covers.
- The support window — expressed as a date or as a rule that resolves to a date.
- What “security update” means for you, and what is out of scope.
- How you announce end of support, and how much notice.
- Where the EOL dates are published.
- How someone reports a security issue to you.
- When you last reviewed it.

Full template in File 04.

## Choosing a window honestly

Pick the number you will actually hit, then publish that. Some anchors:

- **Your longest enterprise contract.** Support cannot end before a contract you have signed commits you.
- **The upstream EOL of your own runtime.** You cannot credibly offer security updates past the point where your platform stops receiving them — unless you are prepared to backport yourself, which is a real cost you should decide on deliberately.
- **Your actual upgrade cadence.** If you have never maintained a branch for more than 18 months, publishing a 5-year window is fiction.

[VERIFY: whether any sector-specific EU or national law imposes a minimum support period on your product – for example medical devices under Regulation (EU) 2017/745, or other sectoral safety law. The PLD imposes none (Recital 51), but other instruments may, and Article 7(2)(f) feeds "relevant product safety requirements, including safety-relevant cybersecurity requirements" back into the defectiveness assessment.]

---

## Artefact 4 — End-of-support communication

The moment your ability to supply updates ends is the moment your factual position changes. Two things follow.

**Tell people, before it happens, in a form you can prove you sent.** Recital 51 puts installation by the owner outside your control; that argument is considerably easier to make when you can show what you told them and when.

**Archive the notice.** A published page that changes silently is weak evidence. Keep a

dated, immutable copy — a git commit of the notice, the sent email with headers, the in-product banner's release tag.

Minimum contents: what is ending, exact date, what stops, what to move to, the risk of staying, who to contact. Template in File 04.

For products you cannot reach — shipped software, air-gapped installs, abandoned-account SaaS tenants — record what channels you attempted and when. The record of a reasonable attempt is the artefact.

---

## Artefact 5 — Recording a decision NOT to update

This is the artefact almost nobody keeps, and the one that most changes how a file reads two years later.

Not every unpatched component should be patched. Sometimes the fix is not reachable, sometimes the upgrade is riskier than the bug, sometimes there is no fix. Those can be entirely sound engineering decisions. **The problem is never the decision; it is that the decision was never recorded, so afterwards it cannot be distinguished from neglect.**

### When to write one

- A component is past EOL and you are staying on it.
- A known vulnerability exists and you are not applying the fix.
- You are ending support for a version earlier than your normal window.
- You are shipping with a dependency you know is unmaintained.

### What it must contain

1. What the component is, and its exact version.
2. What the risk is, in plain terms.
3. **Whether it is reachable in your product, and how you determined that.**
4. Why you are not updating — the real reason, including “no upstream fix exists” and “the migration cost exceeds the assessed risk”.
5. What you are doing instead — mitigation, compensating control, monitoring.
6. What would change the decision, and when you will look again.
7. Who decided, who reviewed, and the date.

## Write it as if it will be read by someone hostile, later

Because it might be. Two rules that follow from that:

- **Do not editorialise and do not speculate about legal consequences.** State facts and engineering reasoning. Speculating in writing about whether something makes you liable helps nobody.
- **Do not write it dishonestly.** A record that minimises a risk you actually understood is far worse than no record. If the honest reason is “we did not have the engineering capacity this quarter”, write that. It is a normal business fact.

[VERIFY: whether internal risk records of this kind attract legal professional privilege in your jurisdiction, and whether they should be routed through counsel to do so. Article 9(4) requires courts to consider legitimate interests including confidentiality, and Recital 42 names legal professional privilege and trade secrets – but privilege is a matter of national law and varies widely across Member States. Ask your lawyer how to structure these records **\*\*before\*\*** you accumulate two years of them.]

That [VERIFY] is the most commercially significant one in this file. It is worth one conversation with counsel before you start.

Template in File 04.

---

## Retention: how long to keep all of this

Set retention against the liability periods, not against your default log retention.

| Period                         | Length                                                                         | Source            |
|--------------------------------|--------------------------------------------------------------------------------|-------------------|
| Limitation                     | 3 years from claimant’s awareness of damage, defectiveness and liable operator | Art. 16(1)        |
| Expiry                         | <b>10 years</b> from placing on market / putting into service                  | Art. 17(1)        |
| Expiry, latent personal injury | <b>25 years</b>                                                                | Art. 17(2)        |
| Substantially modified product | New 10 years from when the modified product was made available                 | Art. 17(1)<br>(b) |

A product placed on the EU market on 9 December 2026 can be the subject of a claim until **9 December 2036**, and in a latent personal injury case until **9 December 2051**.

### **Practical guidance, offered as engineering judgement rather than legal advice:**

- Keep release-linked evidence — inventory, EOL scan, support policy as it stood, end-of-support notices — for at least **10 years from the date that version was placed on the market**.
- Where your product could contribute to personal injury, discuss the **25-year** case with your lawyer before setting a retention period. Twenty-five years is longer than most storage formats and most companies. Plan for format migration.
- Keep the evidence **immutable and dated**. Signed release artefacts, git tags, write-once storage. A record that could have been edited later is weaker than one that could not.
- **Reconcile this with data protection retention rules**. These artefacts should contain no personal data beyond decision-maker names; keep it that way, and check your retention schedule against your privacy policy. Long retention of technical records is not a licence to retain personal data.

[VERIFY: your own retention obligations and limits under national law and your data protection policies. Retention periods are a legal question. The periods above are what the directive states; what you must and may keep is not.]

---

## **The afternoon: a concrete sequence**

For one product, about three hours.

### **0:00 — Baseline scan (15 min)**

```
./pld-watch scan --json > evidence/${date +%Y-%m-%d}-pld-watch.json  
cat evidence/*.json | head -40
```

Commit it. Even if the result is ugly. **Especially** if the result is ugly — a first honest baseline is the point.

**0:15 — Fill the gaps the tool cannot see (45 min)** List, by hand: vendored code, base image OS packages, binaries you ship, native modules, commercial dependencies, anything in unknown. Add them to the inventory with source: manual.

**1:00 — Triage each EOL finding (45 min)** For each, decide one of: **upgrade** (with a target date and an owner), **mitigate** (with the control named), or **accept** (which requires

a decision-not-to-update record). Nothing stays untriaged.

**1:45 — Draft the support policy (45 min)** Template in File 04. Choose a window you will actually hit. Mark it DRAFT and send it to legal review. Do not publish today.

**2:30 — Write the register and any accept-decisions (30 min)** Templates in File 04.

**3:00 — Set the recurring work** - pld-watch in CI on every release build, JSON archived. - Quarterly review of the EOL register. - Annual review of the support policy. - A calendar entry to re-check national transposition law for each Member State you sell into.

---

## What “good” looks like after six months

Not a binder. Four things:

1. Every release has an attached, dated component inventory and EOL scan.
2. Every EOL component is either scheduled, mitigated, or has a written accept decision with a review date.
3. Your public support policy states a window you are actually meeting, and was reviewed by a lawyer before publication.
4. Every end-of-support event has a dated, archived notice.

If an Article 9 disclosure order landed, you would be compiling from records that already exist rather than reconstructing intent from memory. That is the entire objective. It is not a guarantee of anything, and anyone selling it as one is overselling.

---

*Toledo Technologies LLC. Not legal advice. Sources: Directive (EU) 2024/2853 — <https://eur-lex.europa.eu/eli/dir/2024/2853/oj> ; pld-watch — <https://github.com/ntoledo319/pld-watch> ; endoflife.date — <https://endoflife.date>*

## 04 — Support policy templates

---

**Four copy-paste templates: a public security-update support policy, an end-of-support notice, a component end-of-life register, and an internal record for a decision not to patch.**

---

**Not legal advice.** These are drafting starting points written by engineers, not lawyers. **Every template in this file must be reviewed by a qualified lawyer before you publish it or rely on it.** A published support commitment can create contractual and consumer-law obligations that have nothing to do with Directive (EU) 2024/2853, and those vary by Member State. Nothing here limits or excludes liability — Article 15 of the directive does not permit that — and nothing here is a certification or a compliance determination.

---

### How to use these

Fields to fill are marked [LIKE THIS]. Guidance notes are in > blockquotes and **must be deleted before publication**. Every template ends with a review block; fill it in, because an unreviewed, undated policy is weak evidence and an unreviewed public commitment is a commercial risk.

Order of work: Template C (register) first, because it tells you what you can honestly promise. Then Template A (policy). B and D as events require.

---

### Template A — Public security update support policy

**Publish this on a stable, linkable URL.** Link it from your docs, your pricing page and your release notes. Keep every prior version in git — a policy you can prove the state of on a given date is worth considerably more than one you cannot.

**Legal review is mandatory before publication.** This document makes public commitments.

## # Security Update Policy – [PRODUCT NAME]

Last reviewed: [YYYY-MM-DD]

Next scheduled review: [YYYY-MM-DD]

Version: [1.0]

### ## Scope

This policy covers:

- [PRODUCT NAME], versions [X.Y] and later
- [COMPONENT / SDK / AGENT NAME], versions [X.Y] and later

This policy does not cover:

- versions earlier than [X.Y], which are end-of-support (see the register at [URL])
- [BETA / PREVIEW / EXPERIMENTAL FEATURES], which are supplied without a security update commitment
- [THIRD-PARTY INTEGRATIONS / PLUGINS NOT BUILT BY US]
- [SELF-HOSTED DEPLOYMENTS MODIFIED BY THE CUSTOMER]

> Guidance, delete before publishing: be specific about modified deployments.  
> If a customer substantially modifies your product and supplies it onward, they  
> may be treated as a manufacturer of the modified product (Directive (EU)  
> 2024/2853, Art. 8(2)). That is their position to manage, but your policy  
> should be clear about what you are and are not maintaining.

### ## What we mean by a security update

A security update is a change we release to address a vulnerability that could affect the security or safety of the product.

We assess reported and discovered vulnerabilities using [CVSS v3.1 / CVSS v4.0 / OUR STATED SEVERITY SCHEME] and aim to release fixes on these timelines from the point at which we confirm the issue:

| Severity | Target time to fix       |  |
|----------|--------------------------|--|
| ---      | ---                      |  |
| Critical | [e.g. 7 days]            |  |
| High     | [e.g. 30 days]           |  |
| Medium   | [e.g. 90 days]           |  |
| Low      | [next scheduled release] |  |

These are targets, not guarantees. Where we cannot meet a target we will [STATE WHAT YOU DO – e.g. publish an advisory describing the issue and any available mitigation].

> Guidance, delete before publishing: publish targets you actually hit. A missed

> published target is a documented gap between what you said and what you did.  
> If you do not measure your current times to fix, measure for a quarter before  
> you publish a number.

## ## Support window

| Version | Released     | Security updates until | Status                |
|---------|--------------|------------------------|-----------------------|
| [3.x]   | [YYYY-MM-DD] | [YYYY-MM-DD]           | Supported             |
| [2.x]   | [YYYY-MM-DD] | [YYYY-MM-DD]           | Security updates only |
| [1.x]   | [YYYY-MM-DD] | [YYYY-MM-DD]           | End of support        |

Our general rule: we provide security updates for [a major version for N months after the release of its successor / the current and one previous major version / N years from a version's release date].

[FOR SAAS:] This service is operated as a single continuously updated version. Security updates are applied by us to the running service. We do not maintain older versions. Where a change requires action from you, we will tell you [HOW AND HOW FAR IN ADVANCE].

> Guidance, delete before publishing: pick ONE rule and make sure the table  
> matches it. A rule that contradicts your own table is the first thing anybody  
> reviewing this will notice.

## ## End-of-support notice

We will announce the end of security updates for a version at least [N months – commonly 6 or 12] in advance, via:

- [our release notes at URL]
- [email to account administrators]
- [an in-product notice]
- [the end-of-life register at URL]

## ## After end of support

After the date listed above, we do not issue security updates for that version. Continuing to run an unsupported version means known vulnerabilities in it will not be fixed by us. We recommend upgrading to a supported version before the end-of-support date.

## ## Third-party components

[PRODUCT NAME] includes third-party and open-source components. Where an upstream project releases a security fix for a component we ship, we assess it and, where it affects our product, include it in a release on the timelines above.

Where a component we ship has reached its own upstream end of life and no upstream fix is available, we [STATE WHAT YOU ACTUALLY DO – e.g. plan a migration to a supported component, or maintain our own patched build, or document the position in the register at URL].

> Guidance, delete before publishing: only state what you actually do. "We  
> maintain our own patched build" is a serious, ongoing commitment. Do not write  
> it unless someone owns it.

## ## Component inventory

[We publish a software bill of materials (SBOM) for each release at URL.]  
[An SBOM for a given release is available to customers on request at EMAIL.]

## ## Reporting a security issue

Report vulnerabilities to [security@EXAMPLE.COM] [or via our advisory process at URL]. We aim to acknowledge within [N business days].  
[Our PGP key / security.txt is at URL.]

## ## Changes to this policy

We may update this policy. Material changes affecting an existing support window will be announced through the channels listed above. Previous versions are available at [URL / in our public repository].

---

This policy describes our operational practice for security updates. It is not a warranty and does not limit or exclude any rights you have under applicable law.

## Review block — complete before publishing:

Template A review

Drafted by: \_\_\_\_\_ Date: \_\_\_\_\_

Support window verified achievable against:

[ ] longest current enterprise contract commitment

[ ] upstream EOL of our own runtime/platform

[ ] our actual historical maintenance cadence

Severity timelines verified against measured data: [ ] Y [ ] N

Legal review by: \_\_\_\_\_ Date: \_\_\_\_\_

Approved for publication by: \_\_\_\_\_ Date: \_\_\_\_\_

Published at URL: \_\_\_\_\_

## Template B — End-of-support notice

Send this at least the notice period stated in your policy. **Archive a dated copy** — a git commit of the notice, the sent email with headers, the release tag of the in-product banner. A page that can change silently is weak evidence of what you told people and when.

```
# End of security updates for [PRODUCT NAME] [VERSION]
```

```
Published: [YYYY-MM-DD]
```

```
## What is happening
```

```
Security updates for [PRODUCT NAME] [VERSION] end on **[YYYY-MM-DD]**.
```

```
## What that means
```

```
After [YYYY-MM-DD] we will not release security updates for [VERSION]. If a vulnerability is found in [VERSION] after that date, we will not fix it in [VERSION].
```

```
[The version will continue to run / will stop functioning on DATE / will no longer connect to our service from DATE.]
```

```
> Guidance, delete before publishing: state plainly whether the software keeps  
> working. Users conflate "end of support" with "stops working". Ambiguity here  
> is the single most common cause of an unsafe deployment continuing in the  
> field.
```

```
## What you should do
```

```
Upgrade to [VERSION] or later before [YYYY-MM-DD].
```

```
- Upgrade guide: [URL]  
- Breaking changes: [URL]  
- [Migration tooling: URL]
```

```
If you cannot upgrade by that date, contact [EMAIL] before [YYYY-MM-DD] so we can discuss your options.
```

```
## Why we are doing this
```

```
[STATE THE REAL REASON — e.g. "Version 2.x runs on Node.js 16, which reached its upstream end of life on 2023-09-11 and no longer receives security updates from the Node.js project. We cannot supply security updates for the underlying runtime."]
```

```
> Guidance, delete before publishing: give the real reason, with the real
```

> upstream date. It is more persuasive to users, and it is a more useful record.

## Risk of continuing to run [VERSION]

Running [VERSION] after [YYYY-MM-DD] means any vulnerability discovered in it will not be patched by us. [STATE ANY SPECIFIC KNOWN RISK.] We do not recommend running [VERSION] after that date.

## Questions

[EMAIL] · [SUPPORT URL]

## Archive record — keep with the notice:

Template B despatch record

Version ending support: \_\_\_\_\_ End date: \_\_\_\_\_

Notice published: ☐ release notes URL: \_\_\_\_\_ date: \_\_\_\_\_

☐ email to admins recipients: \_\_\_\_\_ date: \_\_\_\_\_

☐ in-product notice release tag: \_\_\_\_\_ date: \_\_\_\_\_

☐ EOL register updated date: \_\_\_\_\_

Notice period given: \_\_\_\_\_ months (policy requires \_\_\_\_\_)

Customers unreachable / no contact channel: \_\_\_\_\_

Channels attempted for those: \_\_\_\_\_

Immutable copy archived at: \_\_\_\_\_

Recorded by: \_\_\_\_\_ Date: \_\_\_\_\_

## Template C — Component end-of-life register

Internal. One row per component that is past, or approaching, upstream end of life. `pld-watch --json` populates the first six columns; the rest is human judgement that no tool can supply.

Review quarterly. Keep every historical version — the trend over time is itself the record.

# Component End-of-Life Register — [PRODUCT NAME]

Owner: [TEAM / NAME]

Last reviewed: [YYYY-MM-DD] Next review: [YYYY-MM-DD]

Inputs: `pld-watch scan` [YYYY-MM-DD] · SBOM [BUILD ID] · [SCA TOOL] [DATE]  
· manual review [YYYY-MM-DD]

Coverage note: this register covers [WHAT — e.g. declared runtimes and frameworks, direct dependencies, base image packages]. It does NOT currently cover [WHAT — e.g. transitive dependencies beyond depth 1, vendored code]

in /third\_party]. Known gaps are listed at the end.

| #   | Component   | Version | Source       | Upstream EOL       | Status             | Reachable in product? | Decision                  | Target date    | Owner    | Decision record |
|-----|-------------|---------|--------------|--------------------|--------------------|-----------------------|---------------------------|----------------|----------|-----------------|
| --- | ---         | ---     | ---          | ---                | ---                | ---                   | ---                       | ---            | ---      | ---             |
| 1   | Node.js     | 16.20   | .nvmrc       | 2023-09-11         | EOL                | Yes - runtime         | Upgrade to 22 LTS         | 2026-11-30     | platform | -               |
| 2   | next        | 12.3    | package.json | 2022-11-21         | EOL                | Yes - web tier        | Upgrade to 15             | 2027-01-31     | web      | -               |
| 3   | libfoo      | 1.2.0   | vendored     | unknown (archived) | Unknown            | No - build-time only  | Accept                    | review 2027-03 | build    | DNU-2026-004    |
| 4   | [COMPONENT] | [VER]   | [FILE]       | [DATE]             | [EOL/soon/unknown] | [Y/N + why]           | [Upgrade/Mitigate/Accept] | [DATE]         | [TEAM]   | [DNU-ID or -]   |

## ## Status values

- **\*\*EOL\*\*** – past upstream end-of-life date. No upstream security fix is available.
- **\*\*Soon\*\*** – upstream EOL within 180 days.
- **\*\*Unknown\*\*** – could not be matched to a catalogue entry. **\*\*Treat as not checked, not as supported.\*\*** Every Unknown needs a manual determination.

## ## Decision values

- **\*\*Upgrade\*\*** – migrating off it. Needs a target date and a named owner.
- **\*\*Mitigate\*\*** – staying on it with a compensating control. Name the control.
- **\*\*Accept\*\*** – staying on it as-is. **\*\*Requires a decision record (Template D).\*\***

## ## Reachability

State how you determined it, not just the answer. "No - build-time only, not in the shipped artefact" is a record. "No" alone is not.

## ## Known gaps in this register

- [e.g. Transitive npm dependencies are not individually EOL-tracked.]
- [e.g. Base image OS packages are tracked only at image level, not per package.]
- [e.g. Component X's upstream publishes no EOL dates.]

> Guidance, delete before circulating: keep this gaps section honest and  
> current. A register that silently overstates its coverage is worse than one  
> that states its limits.

## ## Change log

| Date         | Change                               | By     |
|--------------|--------------------------------------|--------|
| ---          | ---                                  | ---    |
| [YYYY-MM-DD] | [Added rows 3-4 from pld-watch scan] | [NAME] |

## Template D — Internal record: decision not to update

Internal, retained. One per decision. Write it at the time of the decision, not afterwards.

**Read this first:** discuss with your lawyer, before you accumulate these, whether records of this kind should be created under legal privilege in your jurisdiction, and how. Privilege is a matter of national law and varies widely across Member States. See the [VERIFY] note in File 03.

# Decision not to update – [DNU-YYYY-NNN]

Date of decision: [YYYY-MM-DD]

Product / version affected: [PRODUCT] [VERSIONS]

Component: [NAME] [EXACT VERSION]

Decided by: [NAME, ROLE]

Reviewed by: [NAME, ROLE]

Next review date: [YYYY-MM-DD]

## 1. What the component is

[What it is, what it does in our product, how it got here – direct dependency, transitive, vendored, part of a base image.]

## 2. What the issue is

[ ] Component is past upstream end of life – EOL date [YYYY-MM-DD], source [endoflife.date / vendor page URL]

[ ] Known vulnerability – [CVE-XXXX-NNNN], CVSS [SCORE], [LINK]

[ ] Upstream is unmaintained / archived – [EVIDENCE, URL, DATE CHECKED]

[ ] Other: [DESCRIBE]

[Plain description of the risk if it were exploited.]

## 3. Is it reachable in our product?

Reachable: [ ] Yes [ ] No [ ] Partially [ ] Could not determine

How we determined this:

[e.g. "Call graph analysis shows the affected parser is only invoked by the CLI import path, which is not present in the shipped server artefact.

Verified by NAME on DATE against build BUILD-ID."]

> Guidance: "could not determine" is an acceptable and honest answer. It should

> normally push the decision toward Mitigate or Upgrade rather than Accept.

## 4. What updating would involve

[Concretely: what has to change, estimated effort, what would break, why it is not trivial. If no upstream fix exists, say so – that is the whole answer to this section.]

### ## 5. Why we are not updating

- [ ] No upstream fix exists and none is expected
- [ ] Not reachable in our product (see 3)
- [ ] The upgrade carries a higher assessed risk than the issue
- [ ] Blocked by [DEPENDENCY / CUSTOMER / CONTRACT / PLATFORM CONSTRAINT]
- [ ] Engineering capacity – scheduled for [WHEN]
- [ ] Other: [DESCRIBE]

[Reasoning in plain terms.]

- > Guidance: write the real reason. "Engineering capacity, scheduled for Q2" is a
- > normal business fact and a perfectly respectable entry. A record that
- > minimises a risk you actually understood is far worse than no record at all.
- > Do not speculate in this document about legal consequences or liability –
- > state facts and engineering reasoning only.

### ## 6. What we are doing instead

- [ ] Compensating control: [DESCRIBE – WAF rule, network isolation, input validation, feature disabled, config hardening]
- [ ] Monitoring: [WHAT WE WATCH AND WHERE IT ALERTS]
- [ ] Documented for customers: [WHERE]
- [ ] Nothing – accepted as-is

[Detail. Name the specific control, not the category.]

### ## 7. What would change this decision

[e.g. "An upstream fix being released"; "the component becoming reachable from the network tier"; "a public exploit appearing"; "the planned v4 refactor removing this dependency".]

### ## 8. Evidence attached

- [ ] pld-watch scan JSON: [FILENAME / ARTEFACT ID], generated [DATE]
- [ ] SBOM for affected release: [BUILD ID]
- [ ] Vendor / upstream EOL page: [URL], checked [DATE]
- [ ] Vulnerability advisory: [URL]
- [ ] Reachability analysis: [LINK / TICKET]
- [ ] Ticket: [ID]

### ## 9. Review

| Date | Reviewer | Outcome |
|------|----------|---------|
|------|----------|---------|

```
|---|---|---|
| [YYYY-MM-DD] | [NAME] | [Decision stands / changed to X / escalated] |
```

Retention: keep for at least 10 years from the date the affected version was placed on the market (Directive (EU) 2024/2853, Art. 17(1)), and see File 03 on the 25-year latent personal injury case.

---

## Bonus — Supplier identification record (distributors and resellers)

Only relevant if you distribute or resell other people's software into the EU. Under Article 8(3), a distributor becomes liable where an injured person asks it to identify an EU-established economic operator (or its own supplier) and it **fails to do so within one month**. This register is how you answer inside a month without a fire drill.

# Supplier identification register

Last reviewed: [YYYY-MM-DD]

```
| Product we distribute | Manufacturer (legal name) | Established in EU? | EU-established
operator (importer / auth. rep.) | Their registered address | Contact | Our supplier (if not
the manufacturer) | Verified |
|---|---|---|---|---|---|---|---|
| [PRODUCT] | [LEGAL ENTITY] | [Y/N] | [ENTITY NAME] | [ADDRESS] | [EMAIL] | [ENTITY] |
[DATE] |
```

Escalation: a request under Art. 8(3) must be answered within one month.  
Route to: [NAME / ROLE], backup [NAME / ROLE].

Verify entries [annually / on contract renewal]. An out-of-date register does not help you inside a one-month window.

---

## Before any of this goes out

Pre-publication checklist

```
[ ] All [PLACEHOLDERS] filled
[ ] All "> Guidance" blockquotes deleted from public documents
[ ] Support window verified achievable, not aspirational
[ ] Severity timelines match measured reality
[ ] Version table matches the stated support rule
[ ] URLs resolve
[ ] Reviewed by a qualified lawyer in [JURISDICTION]   date: _____
[ ] Owner assigned for the next review               date: _____
```

*Toledo Technologies LLC. Not legal advice. These templates must be reviewed by a qualified lawyer before publication or reliance. Source: Directive (EU) 2024/2853 — <https://eur-lex.europa.eu/eli/dir/2024/2853/oj>*

## 05 — Board brief

---

### EU Product Liability Directive (EU) 2024/2853 — what a board or insurer needs to know.

Prepared by: [NAME, ROLE] · Date: [YYYY-MM-DD] · For: [BOARD / INSURER]

---

**Not legal advice.** This brief summarises a legal instrument for a business audience. It is not legal advice, not a compliance determination, and not a legal opinion on the company's exposure. It was prepared from the primary text on EUR-Lex by the engineering function, not by counsel. Where the position is genuinely unsettled it says so. Take advice from a qualified lawyer before acting on it.

---

#### 1. The change, in five lines

- Directive (EU) 2024/2853 replaces the 1985 product liability directive. It applies to products placed on the EU market or put into service **from 9 December 2026**. Products placed before then stay under the old regime. (Arts. 2(1), 21, 22(1); Art. 2(1) as corrected by Corrigendum 2026/90364.)
- **Software, SaaS and AI systems are now expressly “products”**. (Art. 4(1); Recital 13.)
- Liability is **no-fault**, as it was in 1985. A claimant proves defect, damage and causation — not negligence. (Art. 10(1).)
- **The defence that “the defect arose after we shipped” is switched off** where the defectiveness is due to software, updates, **a lack of updates necessary to maintain safety**, or a substantial modification, and the product is within our control. We are treated as retaining control simply by **having the ability** to supply updates. (Arts. 11(2), 4(5)(b).)
- There are **new procedural mechanisms**: a court can order us to disclose evidence, and failing to disclose **presumes the product defective**. (Arts. 9, 10(2) (a).)

## 2. What it is not — read this before reacting

This is the part most external commentary gets wrong, and a board should hear it first.

- **It is not an emergency.** The directive has been in force since December 2024 and applies from December 2026. This is planned work, not incident response.
- **It is not a certification or registration regime.** There is no badge, no notified body, no audit, no regulator that approves us and no filing to make. Nobody can be “PLD certified”. Any vendor selling that is selling something that does not exist.
- **It does not require us to patch anything.** In terms: “This Directive does not impose any obligation to provide updates or upgrades for a product.” (Recital 51.) It attaches a liability consequence to a *defect consisting in* a missing safety-necessary update, which is a different thing. (Other EU law may impose update duties on us; that is a separate question, outside this brief.)
- **It is not a fines regime.** There are no administrative penalties here. The exposure is civil claims for compensation by injured natural persons.
- **It does not create a B2B damages route.** Only **natural persons** can claim (Art. 5(1)).
- **Shipping a fix is not an admission.** A product is not defective merely because a better product or an update came later (Art. 7(3); Recital 35).

## 3. Our exposure, honestly assessed

Only these damage types are recoverable (Art. 6(1)):

| Recoverable                                                             | Not recoverable                                                                  |
|-------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Death or personal injury, incl. medically recognised psychological harm | The defective product itself                                                     |
| Damage to property (no minimum threshold)                               | Property used <b>exclusively</b> for professional purposes                       |
| Destruction or corruption of <b>non-professional</b> data               | Data used for professional purposes, <b>even if not exclusively</b> (Recital 22) |
|                                                                         | Pure economic loss, privacy infringements, discrimination (Recital 24)           |

**What that means for us specifically:** [COMPLETE THIS HONESTLY – the most important paragraph in the brief. Examples of a truthful answer:]

*If pure B2B SaaS:* “Our product is used exclusively by business customers on business-owned equipment. Our realistic failure modes are downtime and financial loss, which are pure economic loss and not recoverable under this directive (Recital 24). Our assessed PLD exposure is low. Our material regulatory exposure remains GDPR and contractual.”

*If consumer data:* “We hold personal, non-professional data for consumers. A defect causing destruction or corruption of that data is squarely within Art. 6(1)(c). Our exposure is real but bounded by the fact that loss is not automatic where data can be restored at no cost (Recital 20), which makes our backup and restore path directly relevant.”

*If software touching hardware:* “Our software controls [X]. A defect could contribute to physical injury. This is the highest-exposure category and we have taken external legal advice.”

**Financial framing:** there are no liability caps. Article 15 prohibits limiting or excluding liability towards the injured person by contract or by national law, and the old option for a national ceiling is gone. **Our standard limitation-of-liability clause does not apply to a PLD claim.**

**Time exposure.** Ten years from placing a product on the market (Art. 17(1)); 25 years for latent personal injury (Art. 17(2)); a fresh 10 years if a product is substantially modified (Art. 17(1)(b)). A product shipped on 9 December 2026 carries a claim window to **December 2036**, and potentially to **December 2051** for latent injury. That drives a records retention decision, not a product decision.

## 4. What it implies for engineering

Modest, and mostly things a well-run team does anyway. The value is that they are now also evidence.

| Work                                                      | Effort                  | Status |
|-----------------------------------------------------------|-------------------------|--------|
| Component inventory per released version (SBOM)           | [X days]                | [ ]    |
| End-of-life tracking for components we ship               | [X days]                | [ ]    |
| Public security update policy with stated support windows | [X days] + legal review | [ ]    |

| Work                                                    | Effort                | Status |
|---------------------------------------------------------|-----------------------|--------|
| End-of-support notice practice                          | [X days]              | [ ]    |
| Written records where we decide not to patch            | ongoing, ~15 min each | [ ]    |
| Retention of the above aligned to the liability periods | [X days]              | [ ]    |

Our initial baseline: [N] components past upstream end-of-life across [M] products, from a scan dated [YYYY-MM-DD]. [Of those, N are reachable in shipped artefacts.]

[Note the method and its limits: "Scanned with pld-watch against the public endoflife.date catalogue. This covers declared runtimes and frameworks; it is not a vulnerability scan and does not cover transitive dependencies. N components returned 'unknown' and are being determined manually."]

**Estimated cost:** [X engineering days] for the initial pass, then roughly [Y days per quarter] to maintain. [Plus legal review of the published policy, estimated £/€ Z.]

## 5. Why this is worth doing now rather than later

Three reasons, none of them alarmist:

1. **Records cannot be backdated.** A dated component inventory and EOL scan attached to each release is trivial to produce today and impossible to recreate in 2029. Article 9 lets a court order us to disclose evidence, and Recital 42 confirms that can include documents we have to compile from scratch. Compiling from records that exist is a different exercise from reconstructing intent from memory.
2. **Failing to produce evidence has a defined consequence.** Article 10(2)(a): failure to disclose relevant evidence when ordered means defectiveness is presumed. That is the one procedural risk squarely within our control.
3. **Customers are already asking.** [True? If enterprise procurement or insurers have raised this, say so here with specifics. If not, delete this point rather than inventing demand.]

## 6. Decisions we are asking the board for

[EDIT TO WHAT YOU ACTUALLY NEED. Suggested:]

1. **Approve** [X] engineering days this quarter for the component inventory and EOL baseline.

2. **Approve** legal review of a public security update support policy before publication, estimated [cost].
3. **Note** that we will publish stated support windows, which are public commitments we must then meet.
4. **Note** the records retention implication: release-linked technical records kept for 10+ years, which is longer than our current default.
5. [If exposure is in the personal injury category:] **Approve** external legal advice on scope and on insurance adequacy.

## 7. Insurance

[FOR AN INSURER AUDIENCE, OR A BOARD DISCUSSION WITH A BROKER. Questions worth putting to the broker – we are not stating the answers, because coverage is policy-specific:]

- Does our current product liability or tech E&O policy respond to a claim under national law transposing Directive (EU) 2024/2853?
- Does it cover **software as a product**, as opposed to professional services?
- Does it respond to **destruction or corruption of a consumer's data** (Art. 6(1)(c)) as property-type damage?
- Does the policy period model work against a **10-year** (or 25-year) expiry period, and what happens to claims after we stop renewing?
- Does it cover **defence and disclosure costs** in an Article 9 disclosure process?
- Are there exclusions for known-unpatched components or for end-of-life software?

[VERIFY: all of the above with our broker. We are not in a position to state what any policy covers, and this brief does not.]

The Commission is required to report on the availability of product liability insurance in its first evaluation, due by 9 December 2030 (Art. 20). Market practice for software cover is therefore likely to move over the next few years.

## 8. Open questions we cannot answer internally

Stated plainly, because a brief that hides its uncertainty is not useful.

1. **Does a continuously deployed SaaS product get re-placed on the market with each release?** If yes, a long-lived service progressively comes under the new regime rather than staying under the 1985 one. The directive does not address this. It is the most commercially significant open question for us. [If not a SaaS business, delete.]
2. **Does adapting a third-party AI model amount to a “substantial**

**modification”?** If so, the adapter becomes a manufacturer of the modified product (Art. 8(2)). Recital 40 brings continuous learning of an AI system into the concept but does not address deployer-side adaptation such as fine-tuning or prompt-layer configuration. [If we build on third-party models, this is material to us.]

3. **Where does “software” end and “information” begin?** Recital 13 excludes information and “the mere source code of software” from the product concept, but the operative text does not define the boundary.
4. **How will national transposition differ?** Standard of proof, disclosure procedure, non-material damages and any Article 18 derogation from the development risk defence are set nationally. We have not reviewed the transposition law of [MEMBER STATES].
5. **What will “necessary to maintain safety” mean in practice?** This is the phrase in Article 11(2)(c) that decides how far the narrowed defence reaches. It will be settled by case law, not by text. Article 19 requires Member States to publish final appellate judgments and the Commission to maintain a public database of them; early judgments are worth watching.

**Our position on all five:** we are not guessing. Where an answer is needed for a decision, we will take advice on that specific question rather than adopt a general posture.

## 9. Bottom line

The directive is a real and permanent change to the liability position for software sold into the EU, and the update/control point is genuinely new. It is not an emergency, there is nothing to be certified against, and the work it implies for us is modest and mostly overlaps with ordinary engineering hygiene.

The one thing that is time-sensitive is **the record**, because records cannot be created retrospectively. That is what we are asking to start now.

---

## Appendix — primary sources

- Directive (EU) 2024/2853, full text: <https://eur-lex.europa.eu/eli/dir/2024/2853/oj>
- Consolidated text incorporating the corrigendum: <https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=CELEX:02024L2853-20241118>
- Corrigendum 2026/90364 of 7 May 2026 (amends Art. 2(1)): [https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=OJ:L\\_202690364](https://eur-lex.europa.eu/legal-content/EN/TXT/HTML/?uri=OJ:L_202690364)
- Council Directive 85/374/EEC (repealed from 9 December 2026): <https://eur->

## Key provisions cited in this brief

| Provision    | Subject                                                                                 |
|--------------|-----------------------------------------------------------------------------------------|
| Art. 2(1)    | Applies to products placed on market / put into service after 8 Dec 2026 (as corrected) |
| Art. 2(2)    | FOSS outside commercial activity excluded                                               |
| Art. 4(1)    | “Product” includes software                                                             |
| Art. 4(5)(b) | Control = ability to supply updates                                                     |
| Art. 4(18)   | Substantial modification                                                                |
| Art. 5(1)    | Natural persons only                                                                    |
| Art. 6(1)    | Recoverable damage, incl. non-professional data                                         |
| Art. 7(2)    | Defectiveness factors                                                                   |
| Art. 8       | Liable economic operators                                                               |
| Art. 9       | Disclosure of evidence                                                                  |
| Art. 10      | Burden of proof and presumptions                                                        |
| Art. 11(1)   | Defences                                                                                |
| Art. 11(2)   | Narrowing of the later-defect defence                                                   |
| Art. 12(2)   | Recourse waiver for micro/small software component makers                               |
| Art. 15      | No contractual limitation of liability                                                  |
| Arts. 16, 17 | Limitation (3 yrs) and expiry (10 / 25 yrs)                                             |
| Art. 19      | Judgment publication and Commission database                                            |

| Provision | Subject |
|-----------|---------|
|-----------|---------|

|             |                                                    |
|-------------|----------------------------------------------------|
| Art. 21, 22 | Repeal of 85/374/EEC and transposition, 9 Dec 2026 |
|-------------|----------------------------------------------------|

---

*Toledo Technologies LLC. Not legal advice.*